Universida de Vigo

Grado en Inteligencia Artificial
Recuperación de información
Entregable 1
Milagros Fernández Gavilanes • Octubre 2025

1. Descripción general

El objetivo de este trabajo es desarrollar, por grupos de dos estudiantes, un **motor de búsqueda funcional** basado en el motor de indexación. El sistema deberá ser capaz de **parsear**, **indexar** y **consultar** un conjunto de documentos procedentes de la Agencia EFE en formato SGML, que contienen noticias en castellano.

El proyecto permite aplicar los conceptos de:

- Representación documental,
- Extracción y normalización de información textual,
- Indexación inversa, y

2. Conjunto de datos

Los datos que se encuentran en la carpeta `efe.zip` provienen de una colección de documentos de la Agencia EFE en formato SGML. Cada documento está delimitado por las etiquetas `<DOC>` y `</DOC>` y contiene varias etiquetas internas.
La explicación de esta etiquetas es la siguiente:

Etiqueta	Descripción
`<DOCNO>`	Identificador único del documento dentro de la colección (ej. `EFE19940102-00464`).
`<DOCID>`	Identificador alternativo o redundante, suele coincidir con `<DOCNO>`.
`<DATE>`	Fecha de publicación, formato AAAAMMDD (ej. 19940102).
`<TIME>`	Hora de publicación, formato 24h (ej. 23.49).
`<SCATE>`	Subcategoría o código interno (ej. `SUC`).
`<FICHEROS>`	Identificador de fichero multimedia asociado (ej. imagen).
`<DESTINO>`	Códigos de distribución editorial interno (ej. `AMT EXG`).
`<CATEGORY>`	Categoría temática principal de la noticia (ej. `SUCESOS`).
`<CLAVE>`	Código de referencia interno de la noticia.
`<NUM>`	Número secuencial o código de control.
`<PRIORIDAD>`	Indicador de prioridad informativa (ej. `R`).
`<TITLE>`	Título de la noticia.
`<TEXT>`	Cuerpo completo del texto noticioso, sobre el cual se realizará la indexación y búsqueda.

3. Requisitos del sistema a desarrollar

3.1 Parseo de los documentos

- Implementar `parser_efe_corpus.py` que será un parser en Python al que se le pasará la ruta de la carpeta de los archivos SGML y extraerá correctamente la información de cada etiqueta.

- Almacenar los documentos en una estructura manejable (ej. `pandas.DataFrame` o lista de diccionarios).
- Limpiar el texto de saltos de línea y espacios innecesarios.

3.2 Generación de palabras clave

- Cada documento debe ser enriquecido con un conjunto de palabras clave generadas automáticamente mediante `RAKE NLTK` [1]. `RAKE` (*Rapid Automatic Keyword Extraction*) es un algoritmo de extracción automática de palabras clave basado en la identificación de frases frecuentes que no contienen stopwords. Por defecto, la implementación de `RAKE NLTK` está pensada para inglés y utiliza listas de stopwords en inglés.
- Usar stopwords en español para el correcto funcionamiento del algoritmo.
- Existirá una función en `parser_efe_corpus.py` llamada `loadKeywords()` que extraerá automáticamente las palabras clave del documento analizado.
- Solo se permiten como máximo 3 palabras clave por texto.
- Estas palabras clave se almacenarán en un campo adicional del índice (ej. KEYWORDS), estarán separadas por "," y se incluirán en las búsquedas.

3.3 Creación del índice

- Definir un esquema de indexación, justificando qué campos se almacenan y/o indexan.
- Configurar *analyzers* adecuados.
- Implementar un script `efe_indexing.py` que permita:
  - con la opción `--create`, creará un índice desde cero en la ruta indicada en `--path <path>` cargando los documentos en `--documents <SGML files>`. Si el índice ya existe, se eliminará,
  - con la opción `--add`, abrirá el indice indicado en `--path <path>` y añadirá los documentos `--documents <SGML files>`. Si el índice no existe, saldrá, y
  - **(opcional) Consultar estadísticas básicas del índice.**
- Implementar `efe_query.py`: Script vacío para realizar búsquedas sobre un índice

3.4 Búsqueda y recuperación

Implementar una interfaz de búsqueda por consola que permita:

- Consultas en texto libre.
- Permita indicar en qué campos buscar.
- Filtrado opcional por categoría, keyword o fecha.

Al finalizar la consulta, siempre tiene que indicar cuantos resultados hay en total.

(opcional) Mostrar los términos más frecuentes para los diferentes campos indexados.

3.5 Ejemplo de ejecución

$$\text{python efe_indexing.py} \; --\text{create} \; --\text{index indice_efe} \; --\text{documents efe.zip}$$

[1] Disponible en `https://csurfer.github.io/rake-nltk/_build/html/index.html`

4. Entregables

4.1 Código fuente

Se aportará todo el código que conforme el proyecto incluyendo:

- `parser_efe_corpus.py`: parser de documentos SGML.
- `efe_indexing.py`: creación y actualización del índice.
- `efe_query.py`: consultas y recuperación de documentos.

4.2 Memoria técnica (máx. 5 páginas)

La memoria técnica debe documentar el diseño y la implementación del motor de búsqueda, justificando las decisiones tomadas y mostrando ejemplos de uso. Se recomienda incluir los siguientes apartados:

1. **Introducción y objetivos.** Breve explicación del propósito y del enfoque que se le ha dado al proyecto. Se debe indicar qué se espera lograr con el motor de búsqueda y cómo se va a evaluar.
2. **Diseño del sistema.** Descripción detallada del motor de búsqueda:
   - Campos indexados y almacenados.
   - Analizadores utilizados y preprocesamiento de texto realizado (limpieza, normalización, stemming/lemmatización, eliminación de stopwords).
   - Integración de palabras clave generadas automáticamente (RAKE NLTK) y cómo se utilizan en las búsquedas.
   - Justificación de cada elección técnica.
3. **Descripción de la interfaz de búsqueda.** Explicación de cómo el usuario puede realizar consultas:
   - Consultas en texto libre.
   - Posibles filtros por categoría, keyword o fecha.
   - Presentación de resultados: campos mostrados.
4. **Ejemplos de consultas y resultados.** Mostrar ejemplos reales de búsquedas sobre el índice, incluyendo:
   - Consulta utilizada.
   - Documentos recuperados.
5. **Conclusiones y mejoras posibles.** Reflexión sobre el funcionamiento del sistema, dificultades encontradas y posibles mejoras futuras (optimización de RAKE, ampliación de campos, mejoras en la interfaz, etc.).

5. Criterios de evaluación

Aspecto	Ponderación
Parseo y limpieza correcta de los datos SGML	25 %
Definición adecuada del esquema de indexación	25 %
Implementación del sistema de búsqueda y relevancia	25 %
Calidad, claridad y justificación de la memoria técnica	20 %
Organización, documentación y limpieza del código	5 %

6. Entrega

- Archivo comprimido (`ZIP/TAR.GZ`) con:
  - Código fuente.
  - Memoria técnica en PDF.
- Nombre del archivo: `E1_G1|G2_Grupo_ApellidoAlumno1_ApellidoAlumno2.zip`